UNIVERSITÀ DEGLI STUDI DI UDINE

Laboratorio di Programmazione Web
(laurea in Scienze e Tecnologie Multimediali)

A.A. 2026/2027

Web link alle pagine del corso:

http://elearning.uniud.it/moodle/course/view.php?id=6288

WEB technologies and languages:
XML
DTD
(X)HTML

Autore:

Carlo Drioli

I materiali in queste slide sono tratti in parte da:

V. Della Mea, L. Di Gaspero, I. Scagnetto,"Programmazione lato server", Apogeo.

http://www.w3.org/

Wikipedia

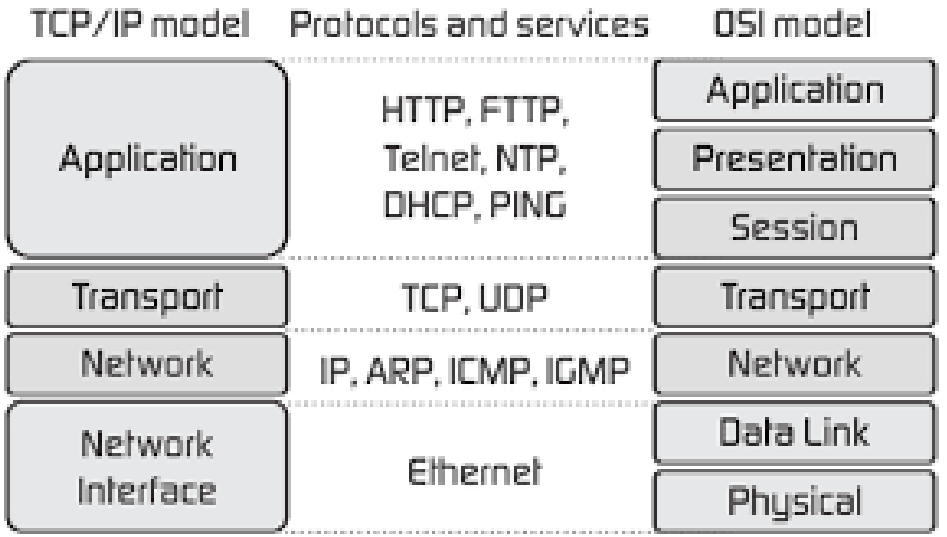
WEB: technologies and languages

Client – Server communication: the HTTP protocol

HTTP (Hypertext Transfer Protocol) is the set of rules for transferring files, such as text, graphic images, sound, video, and other multimedia files, on the World Wide Web

Web browser are HTTP clients, sending requests to server machines. Browser users can enter file requests by either "opening" a web file (typing in a URL) or clicking on a hypertext link: the browser then builds an HTTP request and sends it to the Internet

HTTP is an application protocol that runs on top of the TCP/IP suite of protocols



WEB: technologies and languages (client side)

I. Browsers - the web clients

The browsers are clients for accessing the resources in the web.

Usually seen as a display device, with a graphic interface and a point&click interaction paradigm

II. Browser functions

Interpretation of user requests

E.g., access to a particular Web resource, by either writing a resource identifier, or by activating a link on a web page

Resource recovery

Based on the identifier indicated by the user, the browser is able to start a transaction with the server that hosts the resource by sending the HTTP command to demand it, and waiting for the answer, which will include the requested resource.

Resource management.

The browser activates the appropriate actions to manage the resource (e.g., display text/images, render audio/video streams).

WEB: technologies and languages (client side)

II. Browser functions

To manage resources, the browser may use:

internal code:

e.g., HTML for web pages, JPEG, GIF, PNG for images

plugins:

external modules which are executed within the browser interface itself. Often used for audio video formats decoding (MPEG, Real Player, Windows Media, Quicktime, Flash Player)

helper:

external programs that are activated automatically when a resource is requested whose type is associated to a particular program (PDF files, .doc files, etc)

WEB: technologies and languages (client side)

III. Client side programming

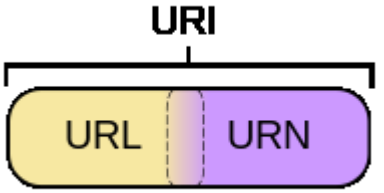
Browsers may execute programs received (as a resource) from remote hosts To this aim, different tools may be used:

Scripting: interpreted languages that allow to program personalized actions of the browser. Most common client-side scripting language today is *Javascript*.

Java Applets: Java programs, compiled in Java byte code, that can be executed by a *Java virtual machine* on any computer system.

Flash applications: a compiled language oriented to multimedia applications. Flash applications require a browser plugin to run on the client host. Allows to create interactive, animated web pages.

WEB: technologies and languages (client side)



IV: links to resources (URN URL URI)

- URI (Uniform Resource Identifier): it represents a resource as well as its location.

- URL (Uniform Resource Locator): it specifies the location of a Web resource with access method (protocol).

  e.g. http://www.animals.com/zoo/bear

- URN (Uniform Resource Name): refers to the resource without specifying the location. It is a *persistent identifier*.

  e.g. urn:123456:bear doi:10.1594/WDCC/CCSRNIES_SRES_B2

https://www.doi.org/factsheets/DOIIdentifierSpecs.html

WEB: technologies and languages (client side)

IV: links to resources (URL URI URN)

The general form of an URI is:

```
< scheme > : < details with scheme-dependent structure >
```

The specific form of an HTTP resource is:

```
http:// < node >:<port>/<path>?<query>#<fragment>
```

where:

- *node* is the name or IP address of the server hosting the resource
- *path* is the resource path in the server file system (default, *index.html*)
- *query* (optional) contains additional identification information
- *fragment* (optional) contains additional information to access a given part or secondary res.

WEB: technologies and languages (client side)

IV: links to web resources

Examples:

```
http://www.domain.com/
http://www.domain.com/dir1/
http://www.domain.com:8043/dir1/dir2/file.html
http://www.domain.com:8043/dir1/file.html#chapter2
http://www.domain.com:8043/handle.php?variable=value
ftp://ftp.is.co.za/rfc/rfc1808.txt
mailto:John.Doe@example.com
telnet://192.0.2.16:80/
```

Languages for WEB documents

I. Languages for Web pages design

1. Markup languages: is a modern system for annotating a document in a way that is syntactically distinguishable from the text

   - (X)HTML: provides a presentation semantics, meaning that its specification prescribes how the structured data are to be presented;

   - XML: a meta markup language, allowing to define markup language syntax. Used to define the structure of data contents.

- Cascading Style Sheets (CSS): a style sheet language used for describing the presentation semantics (the look and formatting) of a document written in a markup language

- Data serialization languages: XML, YAML, JSON

Languages for WEB documents

II. Markup Languages

A markup language is a system for annotating text document, in a way that:

1. the content can be structured in logical units called *elements*

1. elements are named using *labels*

1. labels are special sequences of characters called *markup tags.* Tags are distinguished from text by using special characters

The markup is useful for organizing text documents with a structure, in a way similar to what databases do.

Languages for WEB documents

II. Markup Languages

A markup language is a formal system to exchange and publish information in a structured way, consisting of

- a syntax for the tags (special characters that distinguish them);
- the definition of the elements and tags used in a document;
- possibly, a semantics for tags.

Examples of markup languages are:

RTF (Rich Text Format, interchange format for word processor documents),
TeX (language for the typesetting of texts of a scientific nature),
the *HTML* family of web pages composition languages.
XML (actually, a meta markup language, used to define markup languages and data structures)

Languages for WEB documents

III. HTML

HTML (HyperText Markup Language) is a markup language whose elements allow to represent all the features necessary to the composition and visualization of web pages.

An HTML document has two main parts:

- *head*. The *head element* contains title and meta data of a web document.
- *body*. The *body element* contains the information that you want to display on a web page.

```
1  <!DOCTYPE ... >
2
3  <html>
4
5    <head>
6      ...
7      <title> Tite of the
8       Web page
9      </title>
10   </head>
11
12   <body>
13     Body of the Web page
14   </body>
15
16 </html>
```

<html> tag specifies that the document is written in HTML

<title> tag specifies the web page title

<head> tag specifies informations on scripts, styles, meta data, and more.

<body> tag specifies the content of the document.

Languages for WEB documents

III. HTML

Example:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
<html>

 <head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <meta http-equiv="Content-Language" content="en-us">
  <title>Our first web page!</title>
 </head>

 <body>
  <p>Hello World!</p>
 </body>

</html>
```

Languages for WEB documents

IV. From HTML to XHTML

- An HTML document has in itself contents, structure information, presentation information: not necessarily the optimal solution
- Moreover, HTML does not pose hard constraints on syntax (bad formed HTML is still displayed somehow)
- XHTML is HTML redesigned as XML, combining the strengths of HTML and XML
- XML syntax rules are far more rigorous than HTML. As a result, XHTML makes authors work more precisely and XHTML easier to maintain

V. XML: eXtensible Markup Language

XML is a markup language that:

- defines a set of rules for encoding documents in a format that is both human-readable and machine-readable
- allows do define other, more specific, markup languages (e.g., XHTML, SVG, SMIL, WML, etc.): it is, in this sense, a markup *meta-language*

XML is not a replacement for HTML.

XML and HTML were designed with different goals:

XML was designed to structure, transport and store data, with focus on what data is

HTML was designed to display data, with focus on how data looks

VI. XML: an example

```
1  <?xml version="1.0"?>
2  <address-book>
3  <entry>
4    <name>Homer Simpson</name>
5    <tel>+1-039-1234567</tel>
6    <email ref="mailto:homer@springfield.com"/>
7  </entry>
8  <entry>
9    <name>Barney Gamble</name>
10   <address>
11     <street>Moe Street</street>
12     <region>SP</region>
13     <postal-code>99999</postal-code>
14     <locality>Springfield</locality>
15     <country>US</country>
16   </address>
17   <tel preferred="true">+1-039-77777777</tel>
18   <tel>+1-039-77777888</tel>
19   <email ref="mailto:barney@boe.com"/>
20 </entry>
21 </address-book>
```

XML syntax

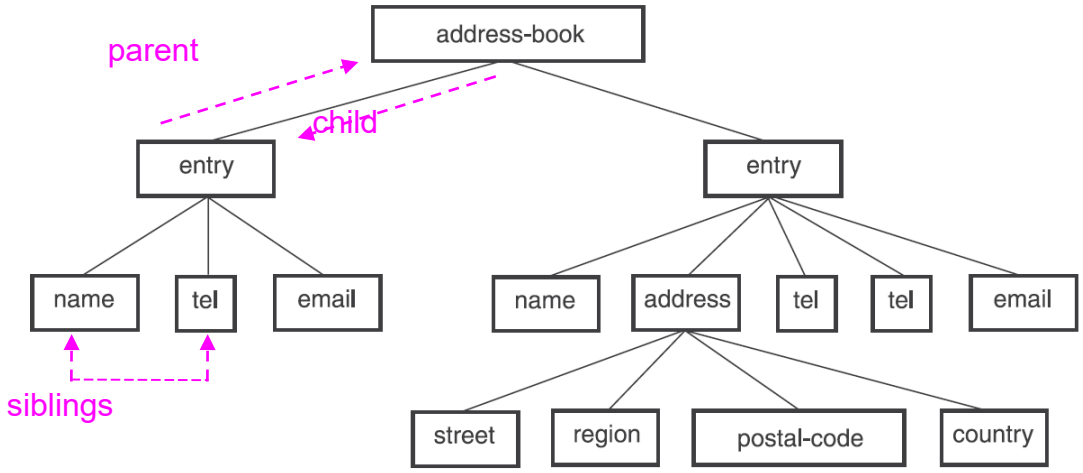
VII. The hierarchical structure of XML documents

Elements in XML documents form a tree structure.

An XML document must contain a root (and one only)

The tree starts at the root and branches to the lowest level of the tree

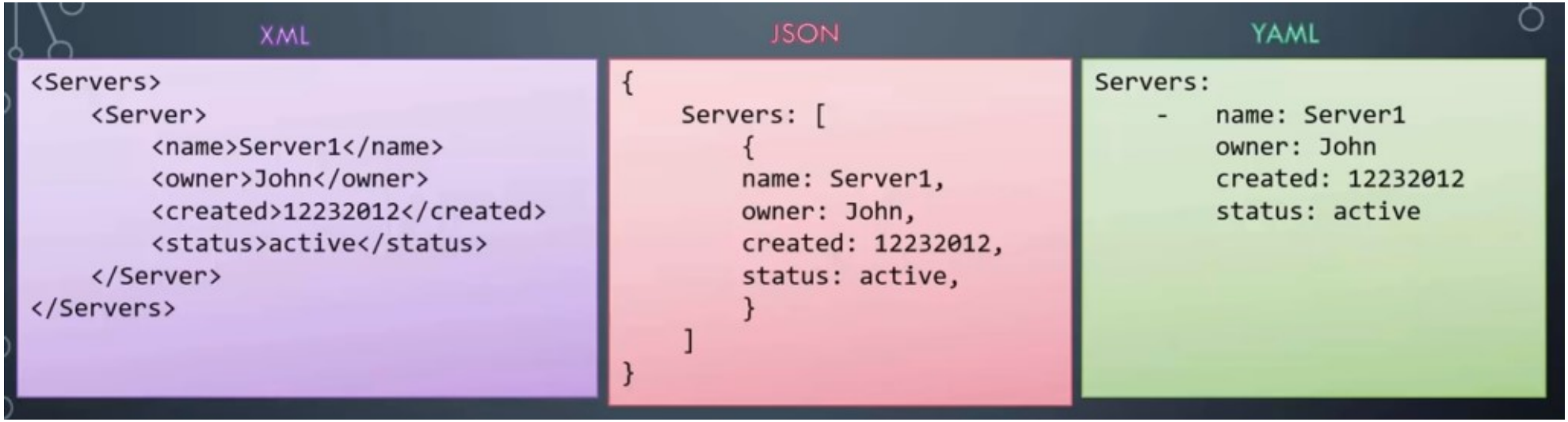
```
1  <?xml version="1.0"?>
2  <address-book>
3   <entry>
4    <name>Homer Simpson</name>
5    <tel>+1-039-1234567</tel>
6    <email ref="mailto:homer@springfield.com"/>
7   </entry>
8   <entry>
9    <name>Barney Gamble</name>
10   <address>
11     <street>Moe Street</street>
12     <region>SP</region>
13     <postal-code>99999</postal-code>
14     <locality>Springfield</locality>
15     <country>US</country>
16   </address>
17   <tel preferred="true">+1-039-77777777</tel>
18   <tel>+1-039-77777888</tel>
19   <email ref="mailto:barney@boe.com"/>
20  </entry>
21 </address-book>
```



XML vs JSON vs YAML syntax

Different solutions to data organization and serialization

- XML is a markup language, whereas JSON and YAML are data formats
- XML uses tags to define the elements and stores data in a tree structure
- data in JSON is stored like a map with key/value pairs
- YAML allows representation of data both in list or sequence format and in the form of a map with key/value pairs



XHTML Text formatting

```
<h2>After Computers</h2>
<p>Programming became <b>significantly</b> more interesting after the computer
was invented...at least until <i>COBOL</i> made it’s way onto the scene.
<big>Hint:</big>
Don’t spend your time learning COBOL right now -- learn C or Java. <small>Your
mileage may vary -- offer not valid in some areas.</small></p>
```



After Computers

Programming became **significantly** more interesting after the computer was invented...at least until *COBOL* made it's way onto the scene.

Hint:

Don't spend your time learning COBOL right now **--** learn C or Java. Your mileage may vary -- offer not valid in some areas.

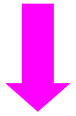
XHTML hyperlinks: the anchor tag

```
1 <a id="qui"/>
2 <p>Il primo link punta al <a href="http://www.uniud.it">sito
3 dell'Universita di Udine</a>; il secondo punta esattamente
4 all'<a href="http://www.dominio.com/pagina.html#qui">inizio
5 </a> di questo esempio.</p>
```



Lists

```
1 <ul>
2  <li>Bart</li>
3  <li>Lisa</li>
4  <li>Maggie (<em>la più giovane</em>)</li>
5 </ul>
```



- Bart
- Lisa
- Maggie (*la più giovane*)

Tables

```
1  <table>
2   <caption>Una tabella semplice</caption>
3   <tr>
4    <th>Prima riga</th>
5    <td>prima cella della prima riga</td>
6    <td>seconda cella della prima riga</td>
7    <td>terza della prima</td>
8   </tr>
9   <tr>
10   <th>Seconda riga</th>
11   <td>prima cella della seconda riga</td>
12   <td>seconda della seconda</td>
13   <td>terza della seconda</td>
14  </tr>
15 </table>
```



Una tabella semplice

Prima riga	prima cella della prima riga	seconda cella della prima riga	terza della prima
Seconda riga	prima cella della seconda riga	seconda della seconda	terza della seconda

Link to Cascading Style Sheets (CSS)

- Modern design of Web documents requires separation of presentation and content
- Content refers to the information as well as its structure.
- Presentation refers to anything related to how the "content" (or structure) is presented, e.g., size, color, margins, borders, …
- Cascading Style Sheets (CSS) is a simple mechanism for adding style (e.g., fonts, colors, spacing) to Web documents.

```
<head>
 <title>Titolo della pagina</title>
 <style type="text/css">
 p {
   color: red;
   font-size: 14pt;
 }
 </style>
 ...
</head>
```

or

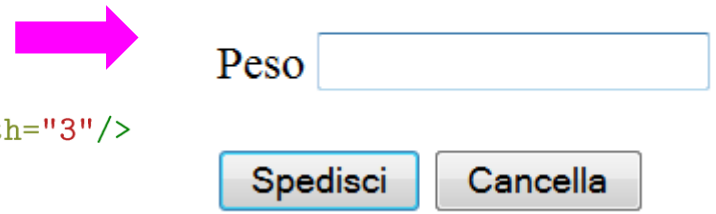
```
<head>
 <title>Titolo della pagina</title>
 <link rel="stylesheet" href="presentazione.css"
  type="text/css" />
</head>
...
```

Input forms

XHTML allows to send user data to server, through *forms* and *controls*

Data must be processed through server-side programming

```
1  <form action="http://www.dominio.com/elabora.cgi" method="post" id="formsemplice">
2   <p>
3    <label for="altezza">Altezza</label>
4    <input type="text" id="altezza" name="altezza"
5    size="10" maxlength="3"/>
6   </p>
7   <p>
8    <label for="peso">Peso</label>
9    <input type="text" id="peso" name="peso" size="20" maxlength="3"/>
10  </p>
11  <p>
12   <input type="submit" value="Spedisci" />
13   <input type="reset" value="Cancella" />
14  </p>
15 </form>
```



(X)HTML: Input form syntax

The *form* element

Form controls are available through the *form* element with the following syntax:

```
<form  attribute_list>
    control_elements
</form>
```

form attributes

- *action :* where to send data when the form is submitted.
- *method ("post"/"get") :* how data must be sent.
- *id :* unique form identifier.

Example

```
<form action="http://www.dominio.com/elabora.php" method="post" id="formsemplice">
  ....
</form>
```

(X)HTML: Input form syntax

Form control elements

- *Edit box* for text input (single line/ multiple lines).
- *Radio box* for single choice selection.
- *Check box* for multiple choice selection.
- *Drop menu* for single/multiple selection.
- *Send/Reset* buttons

The *input* element

< input type =

text	- inserts a text box
checkbox	- inserts a check box
radio	- inserts a radio button
password	- displays chars as "****" to hide input
submit	- submit data
reset	- deletes all entries in the form

name =

"input_variable_name" - used to identify the variable

value =

"default value" - value of the variable (e.g., a text string)

other =

other attribute such as size. Depends on type.

/>

(X)HTML: Input form syntax

Text input box

```
< input
type = “text”
        name = “a_name”
        value = “default value”
        size = “box size (in chars)”
        maxlength = “max n. of chars”
        Id = “an identifier (e.g. to link labels) ”

    />
```

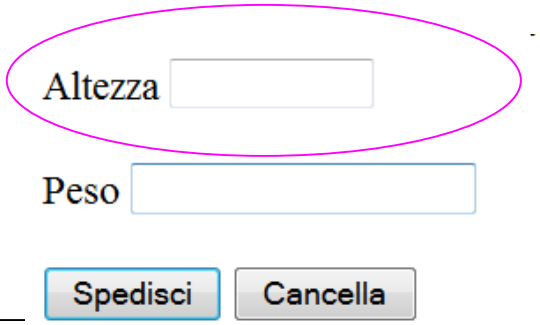
Labels

```
< label for = “a form element identfier” />Label text</label>
```

Example

```
1 <form action="http://www.dominio.com/elabora.cgi" method="post" id="formsemplice">
2  <p>
3   <label for="altezza">Altezza</label>
4   <input type="text" id="altezza" name="altezza"
5   size="10" maxlength="3"/>
6  </p>
7  .....
8 </form>
```





(X)HTML: Input form syntax

Submit/Reset

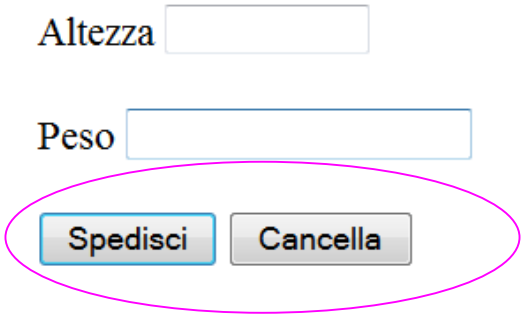
< input
type = “submit”/”reset”

value = “button label”
/>

Example

```
1 <form action="http://www.dominio.com/elabora.cgi" method="post" id="formsemplice">
2 .....
3   <input type="submit" value="Spedisci" />
4   <input type="reset" value="Cancella" />
5 .....
6 </form>
```





(X)HTML: Input form syntax

Radio button

< input
type = “radio”

name = “a_name” (a group of radio buttons
must share the same name)

value = “value to send when checked”
/>

Example

```
<form action="http://www.dominio.com/elabora.cgi" method="post" id="formsemplice">
.....
 <p>
    First period<input type="radio" name="period"  value="First"/>
    Second period<input type="radio" name="period"  value="Second"/>
    Third period<input type="radio" name="period"  value="Third"/>
</p>
.....
</form>
```



Which period?:

First period ◉ Second period ○ Third period ○

Submit Reset

(X)HTML: Input form syntax

Checkbox

< input
type = “checkbox”

name = “a_name” (a group of check boxes
must have different values if
multiple choices are allowed)

value = “value to send when checked”
/>

Example

```
<form action="http://www.dominio.com/elabora.cgi" method="post" id="formsemplice">
.....
 <p>
  <input type="checkbox" name="period[a]" value="First"/>First period
  <input type="checkbox" name="period[b]" value="Second"/>Second period
  <input type="checkbox" name="period[c]" value="Third"/>Third period
</p>
.....
</form>
```



Which period?

First period Second period Third period

Submit Reset

(X)HTML: Input form syntax

Text area

< textarea name = “a_name” cols = “n. of columns” rows = “n. of rows” >

Some initial text

</textarea>



Some initial text

Submit Reset

(X)HTML: Input form syntax

Selection menu

```
< select   name = “a_name” >

  <option value = “menu_entry_name_1”>entry_label_1</option>
  <option value = “menu_entry_name_2”>entry_label_2</option>
  <option value = “menu_entry_name_3”>entry_label_3</option>
  …
</select>
```

